AI Assistant Coding

Assignment 12.5

Name : Varsha **HT. No :** 2303A52494 **Batch:** 50

Task Description #1 (Sorting – Merge Sort Implementation)

• Task: Use AI to generate a Python program that implements the Merge Sort algorithm.

• Instructions:

o Prompt AI to create a function merge_sort(arr) that sorts a list in ascending order.

o Ask AI to include time complexity and space complexity in the function docstring.

o Verify the generated code with test cases.

• Expected Output:

o A functional Python script implementing Merge Sort with proper documentation.

#PROMPT

""" generate a Python program that implements the Merge Sort algorithm. create a function merge_sort(arr) that sorts

a list in ascending order. Include time complexity and space complexity in the function docstring,

Verify the generated code with test cases."""

#CODE

```
def merge_sort(arr):
    """
    Sorts a list in ascending order using the Merge Sort algorithm.
```

```
    Time Complexity: O(n log n) - where n is the number of elements in the list.
    Space Complexity: O(n) - due to the temporary arrays used for merging.
    """
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid])
    right_half = merge_sort(arr[mid:])

    return merge(left_half, right_half)
def merge(left, right):
    """
    Merges two sorted lists into a single sorted list.

    Time Complexity: O(n) - where n is the total number of elements in both
lists.
    Space Complexity: O(n) - due to the temporary array used for merging.
    """
    merged = []
    left_index = right_index = 0

    while left_index < len(left) and right_index < len(right):
        if left[left_index] < right[right_index]:
            merged.append(left[left_index])
            left_index += 1
```

```
        else:
            merged.append(right[right_index])
            right_index += 1

    # If there are remaining elements in left or right, add them to merged
    merged.extend(left[left_index:])
    merged.extend(right[right_index:])

    return merged
# Test cases
if __name__ == "__main__":
    test_cases = [
        [38, 27, 43, 3, 9, 82, 10],
        [5, 2, 9, 1, 5, 6],
        [12, 11, 13, 5, 6, 7],
        [],
        [1],
        [2, 1]
    ]

    for i, test in enumerate(test_cases):
        sorted_arr = merge_sort(test)
        print(f"Test case {i + 1}: {sorted_arr}")
        meaningful_output = sorted(test)
        assert sorted_arr == meaningful_output, f"Test case {i + 1} failed: expected
{meaningful_output}, got {sorted_arr}"
    print("All test cases passed!")
```

#OUTPUT

```
Test case 1: [3, 9, 10, 27, 38, 43, 82]
Test case 2: [1, 2, 5, 5, 6, 9]
Test case 3: [5, 6, 7, 11, 12, 13]
Test case 4: []
Test case 5: [1]
Test case 6: [1, 2]
All test cases passed!
```

Task Description #2 (Searching – Binary Search with AI Optimization)

• Task: Use AI to create a binary search function that finds a target element in a sorted list.

• Instructions:

o Prompt AI to create a function binary_search(arr, target) returning the index of the target or -1 if not found.

o Include docstrings explaining best, average, and worst-case complexities.

o Test with various inputs.

• Expected Output:

o Python code implementing binary search with AI-generated comments and docstrings.

#PROMPT

""" create a python function binary_search(arr,target) returning the index of the target or -1 if not found.

Include docstrings explaining best, average, and worst-case complexities. Test with various inputs."""

#CODE

```
def binary_search(arr, target):
    """
```

```
    Performs binary search on a sorted list to find the index of the target element.

    Time Complexity:
    - Best Case: O(1) - when the target is at the middle of the list.
    - Average Case: O(log n) - where n is the number of elements in the list.
    - Worst Case: O(log n) - when the target is not present in the list or is at one of the ends.

    Space Complexity: O(1) - as we are using a constant amount of space for variables.
    """
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = left + (right - left) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return -1
# Test cases
if _name_ == "_main_":
    test_cases = [
        ([1, 2, 3, 4, 5], 3),  # target in the middle
        ([1, 2, 3, 4, 5], 1),  # target at the beginning
        ([1, 2, 3, 4, 5], 5),  # target at the end
        ([1, 2, 3, 4, 5], 6),  # target not in the list
        ([], 1),               # empty list
        ([1], 1),              # single element list with target
```

```
    ([1], 0)            # single element list without target
]
for i, (arr, target) in enumerate(test_cases):
    result = binary_search(arr, target)
    print(f"Test case {i + 1}: array={arr}, target={target}, result={result}")
    if target in arr:
        expected = arr.index(target)
        assert result == expected, f"Test case {i + 1} failed: expected {expected}, got {result}"
    else:
        assert result == -1, f"Test case {i + 1} failed: expected -1, got {result}"
print("All test cases passed!")
```

#OUTPUT

```
Test case 1: array=[1, 2, 3, 4, 5], target=3, result=2
Test case 2: array=[1, 2, 3, 4, 5], target=1, result=0
Test case 3: array=[1, 2, 3, 4, 5], target=5, result=4
Test case 4: array=[1, 2, 3, 4, 5], target=6, result=-1
Test case 5: array=[], target=1, result=-1
Test case 6: array=[1], target=1, result=0
Test case 7: array=[1], target=0, result=-1
All test cases passed!
```

Task Description #3: Smart Healthcare Appointment Scheduling System

A healthcare platform maintains appointment records containing appointment ID, patient name, doctor name, appointment time, and consultation fee. The system needs to:

1. Search appointments using appointment ID.

2. Sort appointments based on time or consultation fee.

Student Task

- **Use AI to recommend suitable searching and sorting algorithms.**
- **Justify the selected algorithms.**

• **Implement the algorithms in Python.**

#PROMPT

"""healthcare platform containins appointment ID, patient name, doctor name, appointment time,

and consultation fee. Implement a python program that allows users to perform the following operations:

1. Search appointments using appointment ID.

2. Sort appointments based on time or consultation fee.

recommend suitable searching and sorting algorithms.Justify the selected algorithms."""

#CODE

```
class Appointment:

    def _init_(self, appointment_id, patient_name, doctor_name, appointment_time, consultation_fee):

        self.appointment_id = appointment_id

        self.patient_name = patient_name

        self.doctor_name = doctor_name

        self.appointment_time = appointment_time

        self.consultation_fee = consultation_fee

class HealthcarePlatform:

    def _init_(self):

        self.appointments = []


    def add_appointment(self, appointment):

        self.appointments.append(appointment)


    def search_appointment_by_id(self, appointment_id):
```

```
        """
        Searches for an appointment by its ID using linear search.

        Time Complexity: O(n) - where n is the number of appointments.
        Space Complexity: O(1) - as we are using a constant amount of space for variables.
        """
        for appointment in self.appointments:
            if appointment.appointment_id == appointment_id:
                return appointment
        return None

    def sort_appointments_by_time(self):
        """
        Sorts appointments based on appointment time using Merge Sort.

        Time Complexity: O(n log n) - where n is the number of appointments.
        Space Complexity: O(n) - due to the temporary arrays used for merging.
        """
        self.appointments = merge_sort(self.appointments, key=lambda x: x.appointment_time)

    def sort_appointments_by_fee(self):
        """
        Sorts appointments based on consultation fee using Merge Sort.

        Time Complexity: O(n log n) - where n is the number of appointments.
```

```
        Space Complexity: O(n) - due to the temporary arrays used for merging.
        """
        self.appointments = merge_sort(self.appointments, key=lambda x:
x.consultation_fee)
def merge_sort(arr, key):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid], key)
    right_half = merge_sort(arr[mid:], key)

    return merge(left_half, right_half, key)
def merge(left, right, key):
    merged = []
    left_index = right_index = 0

    while left_index < len(left) and right_index < len(right):
        if key(left[left_index]) < key(right[right_index]):
            merged.append(left[left_index])
            left_index += 1
        else:
            merged.append(right[right_index])
            right_index += 1

    merged.extend(left[left_index:])
    merged.extend(right[right_index:])
```

```
    return merged
# Test cases
if _name_ == "_main_":
    platform = HealthcarePlatform()
    platform.add_appointment(Appointment(1, "Alice", "Dr. Smith", "2023-10-01 10:00", 100))
    platform.add_appointment(Appointment(2, "Bob", "Dr. Jones", "2023-10-01 11:00", 150))
    platform.add_appointment(Appointment(3, "Charlie", "Dr. Brown", "2023-10-01 09:00", 120))

    # Search for an appointment
    appointment = platform.search_appointment_by_id(2)
    print(f"Search result for appointment ID 2: {appointment.patient_name if appointment else 'Not found'}")

    # Sort appointments by time
    platform.sort_appointments_by_time()
    print("Appointments sorted by time:")
    for appt in platform.appointments:
        print(f"{appt.patient_name} - {appt.appointment_time}")

    # Sort appointments by fee
    platform.sort_appointments_by_fee()
    print("Appointments sorted by consultation fee:")
    for appt in platform.appointments:
```

```
print(f"{appt.patient_name} - {appt.consultation_fee}")
```

#OUTPUT

```
Search result for appointment ID 2: Bob
Appointments sorted by time:
Charlie - 2023-10-01 09:00
Alice - 2023-10-01 10:00
Bob - 2023-10-01 11:00
Appointments sorted by consultation fee:
Alice - 100
Charlie - 120
Bob - 150
```

Task Description #4: Railway Ticket Reservation System

Scenario

A railway reservation system stores booking details such as ticket ID, passenger name, train number, seat number, and travel date. The system must:

1. Search tickets using ticket ID.

2. Sort bookings based on travel date or seat number.

Student Task

- **Identify efficient algorithms using AI assistance.**
- **Justify the algorithm choices.**
- **Implement searching and sorting in Python.**

#PROMPT

""" A railway reservation system stores booking details such as ticket ID, passenger name, train number, seat number, and travel date. The

Generate a python function that allows users to perform the following operations:

1. Search tickets using ticket ID.

2. Sort bookings based on travel date or seat number.

Recommend suitable searching and sorting algorithms. Justify the selected algorithms."""

#CODE

```
def _init_(self, ticket_id, passenger_name, train_number, seat_number,
travel_date):
        self.ticket_id = ticket_id
        self.passenger_name = passenger_name
        self.train_number = train_number
        self.seat_number = seat_number
        self.travel_date = travel_date
class RailwayReservationSystem:
    def _init_(self):
        self.bookings = []

    def add_booking(self, booking):
        self.bookings.append(booking)

    def search_ticket_by_id(self, ticket_id):
        """
        Searches for a ticket by its ID using linear search.

        Time Complexity: O(n) - where n is the number of bookings.
        Space Complexity: O(1) - as we are using a constant amount of space for
variables.
        """
        for booking in self.bookings:
            if booking.ticket_id == ticket_id:
                return booking
        return None
```

```
    def sort_bookings_by_date(self):
        """
        Sorts bookings based on travel date using Merge Sort.

        Time Complexity: O(n log n) - where n is the number of bookings.
        Space Complexity: O(n) - due to the temporary arrays used for merging.
        """
        self.bookings = merge_sort(self.bookings, key=lambda x: x.travel_date)

    def sort_bookings_by_seat_number(self):
        """
        Sorts bookings based on seat number using Merge Sort.

        Time Complexity: O(n log n) - where n is the number of bookings.
        Space Complexity: O(n) - due to the temporary arrays used for merging.
        """
        self.bookings = merge_sort(self.bookings, key=lambda x: x.seat_number)
def merge_sort(arr, key):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid], key)
    right_half = merge_sort(arr[mid:], key)

    return merge(left_half, right_half, key)
```

```
def merge(left, right, key):
    merged = []
    left_index = right_index = 0

    while left_index < len(left) and right_index < len(right):
        if key(left[left_index]) < key(right[right_index]):
            merged.append(left[left_index])
            left_index += 1
        else:
            merged.append(right[right_index])
            right_index += 1

    # If there are remaining elements in left or right, add them to merged
    merged.extend(left[left_index:])
    merged.extend(right[right_index:])

    return merged
# Test cases
if _name_ == "_main_":
    reservation_system = RailwayReservationSystem()

    # Adding bookings
    reservation_system.add_booking(Booking("T001", "Alice", "TrainA", 12,
"2023-10-01"))
    reservation_system.add_booking(Booking("T002", "Bob", "TrainB", 5, "2023-
09-15"))
```

```
    reservation_system.add_booking(Booking("T003", "Charlie", "TrainA", 8,
"2023-10-05"))


    # Searching for a ticket
    ticket = reservation_system.search_ticket_by_id("T002")
    print(f"Search result for T002: {ticket.passenger_name if ticket else 'Not
found'}")


    # Sorting bookings by travel date
    reservation_system.sort_bookings_by_date()
    print("Bookings sorted by travel date:")
    for booking in reservation_system.bookings:
        print(f"{booking.passenger_name} - {booking.travel_date}")


    # Sorting bookings by seat number
    reservation_system.sort_bookings_by_seat_number()
    print("Bookings sorted by seat number:")
    for booking in reservation_system.bookings:
        print(f"{booking.passenger_name} - Seat {booking.seat_number}")
```

#OUTPUT

```
Search result for T002: Bob
Bookings sorted by travel date:
Bob - 2023-09-15
Alice - 2023-10-01
Charlie - 2023-10-05
Bookings sorted by seat number:
Bob - Seat 5
Charlie - Seat 8
Alice - Seat 12
```

Task Description #5: Smart Hostel Room Allocation System

A hostel management system stores student room allocation details

including student ID, room number, floor, and allocation date. The

system needs to:

1. Search allocation details using student ID.

2. Sort records based on room number or allocation date.

Student Task

- **Use AI to suggest optimized algorithms.**
- **Justify the selections.**
- **Implement the solution in Python.**

#PROMPT

""" Generate a python program for A hostel management system that stores student room allocation details

including student ID, room number, floor, and allocation date. The

system needs to:

1. Search allocation details using student ID.

2. Sort records based on room number or allocation date. Recommend suitable searching and sorting algorithms.

Justify the selected algorithms."""

#CODE

```
def merge_sort(arr, key):
    if len(arr) <= 1:
        return arr


    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid], key)
    right_half = merge_sort(arr[mid:], key)
```

```
    return merge(left_half, right_half, key)
def merge(left, right, key):
    merged = []
    left_index = right_index = 0

    while left_index < len(left) and right_index < len(right):
        if key(left[left_index]) < key(right[right_index]):
            merged.append(left[left_index])
            left_index += 1
        else:
            merged.append(right[right_index])
            right_index += 1

    # If there are remaining elements in left or right, add them to merged
    merged.extend(left[left_index:])
    merged.extend(right[right_index:])

    return merged
class Allocation:
    def _init_(self, student_id, room_number, floor, allocation_date):
        self.student_id = student_id
        self.room_number = room_number
        self.floor = floor
        self.allocation_date = allocation_date
class HostelManagementSystem:
    def _init_(self):
        self.allocations = []
```

```
def add_allocation(self, allocation):
    self.allocations.append(allocation)

def search_allocation_by_student_id(self, student_id):
    """
    Searches for allocation details by student ID using linear search.

    Time Complexity: O(n) - where n is the number of allocations.
    Space Complexity: O(1) - as we are using a constant amount of space for variables.
    """
    for allocation in self.allocations:
        if allocation.student_id == student_id:
            return allocation
    return None

def sort_allocations_by_room_number(self):
    """
    Sorts records based on room number using Merge Sort.

    Time Complexity: O(n log n) - where n is the number of allocations.
    Space Complexity: O(n) - due to the temporary arrays used for merging.
    """
    self.allocations = merge_sort(self.allocations, key=lambda x: x.room_number)

def sort_allocations_by_allocation_date(self):
    """
    Sorts records based on allocation date using Merge Sort.

    Time Complexity: O(n log n) - where n is the number of allocations.
```

```
        Space Complexity: O(n) - due to the temporary arrays used for merging.
        """
        self.allocations = merge_sort(self.allocations, key=lambda x: x.allocation_date)
# Test cases
if _name_ == "_main_":
    hms = HostelManagementSystem()
    hms.add_allocation(Allocation("S001", "101", "1st", "2023-09-01"))
    hms.add_allocation(Allocation("S002", "102", "1st", "2023-09-02"))
    hms.add_allocation(Allocation("S003", "201", "2nd", "2023-09-03"))

    # Test search
    allocation = hms.search_allocation_by_student_id("S002")
    print(f"Search result for student ID 'S002': {allocation.room_number if allocation else 'Not found'}")

    # Test sorting by room number
    hms.sort_allocations_by_room_number()
    print("Allocations sorted by room number:")
    for alloc in hms.allocations:
        print(f"Student ID: {alloc.student_id}, Room Number: {alloc.room_number}")

    # Test sorting by allocation date
    hms.sort_allocations_by_allocation_date()
    print("Allocations sorted by allocation date:")
    for alloc in hms.allocations:
        print(f"Student ID: {alloc.student_id}, Allocation Date: {alloc.allocation_date}")
```

#OUTPUT

```
Search result for student ID 'S002': 102
Allocations sorted by room number:
Student ID: S001, Room Number: 101
Student ID: S002, Room Number: 102
Student ID: S003, Room Number: 201
Allocations sorted by allocation date:
Student ID: S001, Allocation Date: 2023-09-01
Student ID: S002, Allocation Date: 2023-09-02
Student ID: S003, Allocation Date: 2023-09-03
```

Task Description #6: Online Movie Streaming Platform

A streaming service maintains movie records with movie ID, title,

genre, rating, and release year. The platform needs to:

1. Search movies by movie ID.

2. Sort movies based on rating or release year.

Student Task

- **Recommend searching and sorting algorithms using AI.**
- **Justify the chosen algorithms.**
- **Implement Python functions.**

#PROMPT

"""streaming service maintains movie records with movie ID, title, genre, rating, and release year.

Generate a python program that allows users to perform the following operations:

1. Search movies by movie ID.

2. Sort movies based on rating or release year. Recommend suitable searching and sorting algorithms. Justify the selected algorithms."""

#CODE

```
class Movie:

    def _init_(self, movie_id, title, genre, rating, release_year):

        self.movie_id = movie_id

        self.title = title
```

```
        self.genre = genre
        self.rating = rating
        self.release_year = release_year
class StreamingService:
    def _init_(self):
        self.movies = []

    def add_movie(self, movie):
        self.movies.append(movie)

    def search_movie_by_id(self, movie_id):
        """
        Searches for a movie by its ID using linear search.

        Time Complexity: O(n) - where n is the number of movies.
        Space Complexity: O(1) - as we are using a constant amount of space for variables.
        """
        for movie in self.movies:
            if movie.movie_id == movie_id:
                return movie
        return None

    def sort_movies_by_rating(self):
        """
        Sorts movies based on rating using Merge Sort.

        Time Complexity: O(n log n) - where n is the number of movies.
        Space Complexity: O(n) - due to the temporary arrays used for merging.
        """
```

```
        self.movies = merge_sort(self.movies, key=lambda x: x.rating)

    def sort_movies_by_release_year(self):
        """
        Sorts movies based on release year using Merge Sort.

        Time Complexity: O(n log n) - where n is the number of movies.
        Space Complexity: O(n) - due to the temporary arrays used for merging.
        """
        self.movies = merge_sort(self.movies, key=lambda x: x.release_year)
def merge_sort(arr, key):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid], key)
    right_half = merge_sort(arr[mid:], key)

    return merge(left_half, right_half, key)
def merge(left, right, key):
    merged = []
    left_index = right_index = 0

    while left_index < len(left) and right_index < len(right):
        if key(left[left_index]) < key(right[right_index]):
            merged.append(left[left_index])
            left_index += 1
        else:
            merged.append(right[right_index])
```

```
        right_index += 1

    # If there are remaining elements in left or right, add them to merged
    merged.extend(left[left_index:])
    merged.extend(right[right_index:])

    return merged
# Test cases
if _name_ == "_main_":
    streaming_service = StreamingService()
    streaming_service.add_movie(Movie(1, "Inception", "Sci-Fi", 8.8, 2010))
    streaming_service.add_movie(Movie(2, "The Matrix", "Sci-Fi", 8.7, 1999))
    streaming_service.add_movie(Movie(3, "Interstellar", "Sci-Fi", 8.6, 2014))
    streaming_service.add_movie(Movie(4, "The Godfather", "Crime", 9.2, 1972))

    # Search for a movie by ID
    movie = streaming_service.search_movie_by_id(2)
    if movie:
        print(f"Movie found: {movie.title} ({movie.release_year}) - Rating: {movie.rating}")
    else:
        print("Movie not found.")

    # Sort movies by rating
    streaming_service.sort_movies_by_rating()
    print("\nMovies sorted by rating:")
    for movie in streaming_service.movies:
        print(f"{movie.title} ({movie.release_year}) - Rating: {movie.rating}")

    # Sort movies by release year
```

```
streaming_service.sort_movies_by_release_year()
print("\nMovies sorted by release year:")
for movie in streaming_service.movies:
    print(f"{movie.title} ({movie.release_year}) - Rating: {movie.rating}")
```

#OUTPUT

```
ers\shaes\OneDrive\Desktop\Assignments-26\aiac\AIAC-12.5\AIAC-12.5-6.py
Movie found: The Matrix (1999) - Rating: 8.7

Movies sorted by rating:
Interstellar (2014) - Rating: 8.6
The Matrix (1999) - Rating: 8.7
Inception (2010) - Rating: 8.8
The Godfather (1972) - Rating: 9.2

Movies sorted by release year:
The Godfather (1972) - Rating: 9.2
The Matrix (1999) - Rating: 8.7
Inception (2010) - Rating: 8.8
Interstellar (2014) - Rating: 8.6
```

Task Description #7: Smart Agriculture Crop Monitoring System

An agriculture monitoring system stores crop data with crop ID, crop name, soil moisture level, temperature, and yield estimate. Farmers need to:

1. Search crop details using crop ID.

2. Sort crops based on moisture level or yield estimate.

Student Task

- **Use AI-assisted reasoning to select algorithms.**
- **Justify algorithm suitability.**
- **Implement searching and sorting in Python.**

#PROMPT

"""An agriculture monitoring system stores crop data with crop ID, crop name, soil moisture level, temperature, and yield estimate.

Generate a python program that allows users to perform the following operations:

1. Search crop details using crop ID.

2. Sort crops based on moisture level or yield estimate. Recommend suitable searching and sorting algorithms. Justify the selected algorithms."""

#CODE

```
class Crop:
    def __init__(self, crop_id, crop_name, soil_moisture_level, temperature, yield_estimate):
        self.crop_id = crop_id
        self.crop_name = crop_name
        self.soil_moisture_level = soil_moisture_level
        self.temperature = temperature
        self.yield_estimate = yield_estimate
class Agriculture:
    def __init__(self):
        self.crops = []

    def add_crop(self, crop):
        self.crops.append(crop)

    def search_crop_by_id(self, crop_id):
        """
        Searches for crop details by crop ID using linear search.

        Time Complexity: O(n) - where n is the number of crops.
        Space Complexity: O(1) - as we are using a constant amount of space for variables.
        """
        for crop in self.crops:
            if crop.crop_id == crop_id:
```

```
            return crop
        return None

    def sort_crops_by_moisture_level(self):
        """
        Sorts crops based on soil moisture level using Merge Sort.

        Time Complexity: O(n log n) - where n is the number of crops.
        Space Complexity: O(n) - due to the temporary arrays used for merging.
        """
        self.crops = merge_sort(self.crops, key=lambda x: x.soil_moisture_level)

    def sort_crops_by_yield_estimate(self):
        """
        Sorts crops based on yield estimate using Merge Sort.

        Time Complexity: O(n log n) - where n is the number of crops.
        Space Complexity: O(n) - due to the temporary arrays used for merging.
        """
        self.crops = merge_sort(self.crops, key=lambda x: x.yield_estimate)
def merge_sort(arr, key):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid], key)
    right_half = merge_sort(arr[mid:], key)

    return merge(left_half, right_half, key)
```

```
def merge(left, right, key):
    merged = []
    left_index = right_index = 0

    while left_index < len(left) and right_index < len(right):
        if key(left[left_index]) < key(right[right_index]):
            merged.append(left[left_index])
            left_index += 1
        else:
            merged.append(right[right_index])
            right_index += 1

    # If there are remaining elements in left or right, add them to merged
    merged.extend(left[left_index:])
    merged.extend(right[right_index:])

    return merged
# Justification for selected algorithms:
# 1. Linear Search: We use linear search for searching crop details by crop ID because it is simple to implement and works well for small datasets. Since the number of crops may not be very large, linear search is efficient enough for this use case.
# 2. Merge Sort: We use merge sort for sorting crops based on soil moisture level and yield estimate because it is a stable sorting algorithm with a time complexity of O(n log n) in the worst case. Merge sort is efficient for sorting large datasets and can handle different types of data, making it suitable for our crop data. Additionally, merge sort does not require additional space for in-place sorting, which can be beneficial when dealing with larger datasets.
#Test cases
if __name__ == "__main__":
    agriculture_system = Agriculture()

    # Adding sample crops
```

```
agriculture_system.add_crop(Crop(1, "Wheat", 30, 25, 100))
agriculture_system.add_crop(Crop(2, "Corn", 40, 28, 150))
agriculture_system.add_crop(Crop(3, "Rice", 20, 22, 120))

# Searching for a crop by ID
crop = agriculture_system.search_crop_by_id(2)
if crop:
    print(f"Crop found: {crop.crop_name}, Soil Moisture Level: {crop.soil_moisture_level}, Yield Estimate: {crop.yield_estimate}")
else:
    print("Crop not found.")

# Sorting crops by soil moisture level
agriculture_system.sort_crops_by_moisture_level()
print("Crops sorted by soil moisture level:")
for crop in agriculture_system.crops:
    print(f"{crop.crop_name}: {crop.soil_moisture_level}")

# Sorting crops by yield estimate
agriculture_system.sort_crops_by_yield_estimate()
print("Crops sorted by yield estimate:")
for crop in agriculture_system.crops:
    print(f"{crop.crop_name}: {crop.yield_estimate}")
```

#OUTPUT

```
Crop found: Corn, Soil Moisture Level: 40, Yield Estimate: 150
Crops sorted by soil moisture level:
Rice: 20
Wheat: 30
Corn: 40
Crops sorted by yield estimate:
Wheat: 100
Rice: 120
Corn: 150
```

Task Description #8: Airport Flight Management System

An airport system stores flight information including flight ID,

airline name, departure time, arrival time, and status. The system

must:

1. Search flight details using flight ID.

2. Sort flights based on departure time or arrival time.

Student Task

- **Use AI to recommend algorithms.**
- **Justify the algorithm selection.**
- **Implement searching and sorting logic in Python.**

#PROMPT

""" An airport system stores flight information including flight ID, airline name, departure time, arrival time, and status. Generate a python program that allows users to perform the following operations:

1. Search flight details using flight ID.

2. Sort flights based on departure time or arrival time"""”

#CODE

```
class Flight:
    def _init_(self, flight_id, airline_name, departure_time, arrival_time, status):
        self.flight_id = flight_id
        self.airline_name = airline_name
        self.departure_time = departure_time
        self.arrival_time = arrival_time
        self.status = status
class AirportSystem:
    def _init_(self):
        self.flights = []

    def add_flight(self, flight):
        self.flights.append(flight)
```

```
def search_flight_by_id(self, flight_id):
    """
    Searches for flight details by flight ID using linear search.

    Time Complexity: O(n) - where n is the number of flights.
    Space Complexity: O(1) - as we are using a constant amount of space for variables.
    """
    for flight in self.flights:
        if flight.flight_id == flight_id:
            return flight
    return None

def sort_flights_by_departure_time(self):
    """
    Sorts flights based on departure time using Merge Sort.

    Time Complexity: O(n log n) - where n is the number of flights.
    Space Complexity: O(n) - due to the temporary arrays used for merging.
    """
    self.flights = merge_sort(self.flights, key=lambda x: x.departure_time)

def sort_flights_by_arrival_time(self):
    """
    Sorts flights based on arrival time using Merge Sort.

    Time Complexity: O(n log n) - where n is the number of flights.
    Space Complexity: O(n) - due to the temporary arrays used for merging.
    """
```

```
        self.flights = merge_sort(self.flights, key=lambda x: x.arrival_time)
def merge_sort(arr, key):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid], key)
    right_half = merge_sort(arr[mid:], key)

    return merge(left_half, right_half, key)
def merge(left, right, key):
    merged = []
    left_index = right_index = 0

    while left_index < len(left) and right_index < len(right):
        if key(left[left_index]) <= key(right[right_index]):
            merged.append(left[left_index])
            left_index += 1
        else:
            merged.append(right[right_index])
            right_index += 1

    merged.extend(left[left_index:])
    merged.extend(right[right_index:])

    return merged
# Example usage
if _name_ == "_main_":
    airport_system = AirportSystem()
```

```
# Adding some flights
airport_system.add_flight(Flight("FL001", "Airline A", "08:00", "10:00", "On Time"))
airport_system.add_flight(Flight("FL002", "Airline B", "09:00", "11:00", "Delayed"))
airport_system.add_flight(Flight("FL003", "Airline C", "07:30", "09:30", "On Time"))

# Searching for a flight by ID
flight = airport_system.search_flight_by_id("FL002")
if flight:
    print(f"Flight found: {flight.airline_name}, Departure: {flight.departure_time}, Arrival: {flight.arrival_time}, Status: {flight.status}")
else:
    print("Flight not found.")

# Sorting flights by departure time
airport_system.sort_flights_by_departure_time()
print("\nFlights sorted by departure time:")
for flight in airport_system.flights:
    print(f"{flight.flight_id}: {flight.departure_time}")

# Sorting flights by arrival time
airport_system.sort_flights_by_arrival_time()
print("\nFlights sorted by arrival time:")
for flight in airport_system.flights:
    print(f"{flight.flight_id}: {flight.arrival_time}")
```

#OUTPUT

```
Flight found: Airline B, Departure: 09:00, Arrival: 11:00, Status: Delayed

Flights sorted by departure time:
FL003: 07:30
FL001: 08:00
FL002: 09:00

Flights sorted by arrival time:
FL003: 09:30
FL001: 10:00
FL002: 11:00
```